Agent Gateway — Auth Patterns

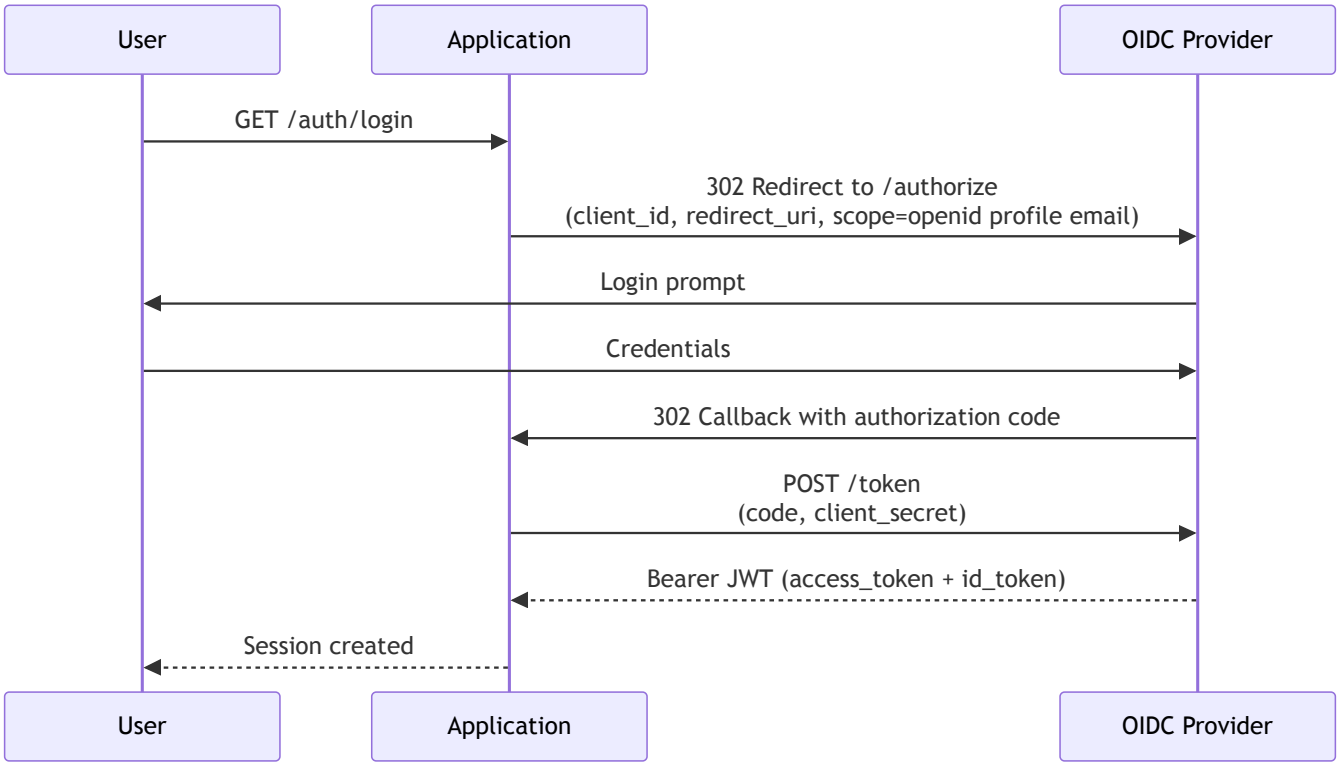
Table of Contents

- Flow 1: Standard OIDC Authentication
- Flow 2a: OBO Delegation (Dual Identity)
- Flow 2b: OBO Impersonation (Token Swap)
- Flow 3: Elicitation (Credential Gathering for Upstream APIs)
- Flow 4: Double OAuth Flow (OIDC Bearer → Upstream Token Exchange)
- Flow 5: Passthrough Token
- Flow 6: Static Secret Injection (Shared Credential)
- Flow 7: Claim-Based Token Mapping (JWT Claim → Static Opaque Token)
- Flow 8: API Key Auth (Inbound)
- Flow 9: Basic Auth (RFC 7617)
- Flow 10: BYO External Auth (gRPC Ext Auth Service)
- Flow 11: MCP OAuth with Dynamic Client Registration
- Flow 12: RBAC Tool-Level Access Control
- Flow 13: Gateway-Mediated OIDC + Token Exchange
- Decision Flowchart: How Should This Request Be Authenticated?

Flow 1: Standard OIDC Authentication

User authenticates via OIDC provider (Authorization Code Flow), receives a bearer JWT, and uses it for all subsequent requests to the Agent Gateway.

> JWT Auth for MCP Services | Set up JWT Auth | Set up Keycloak as IdP
> JWTAuthentication

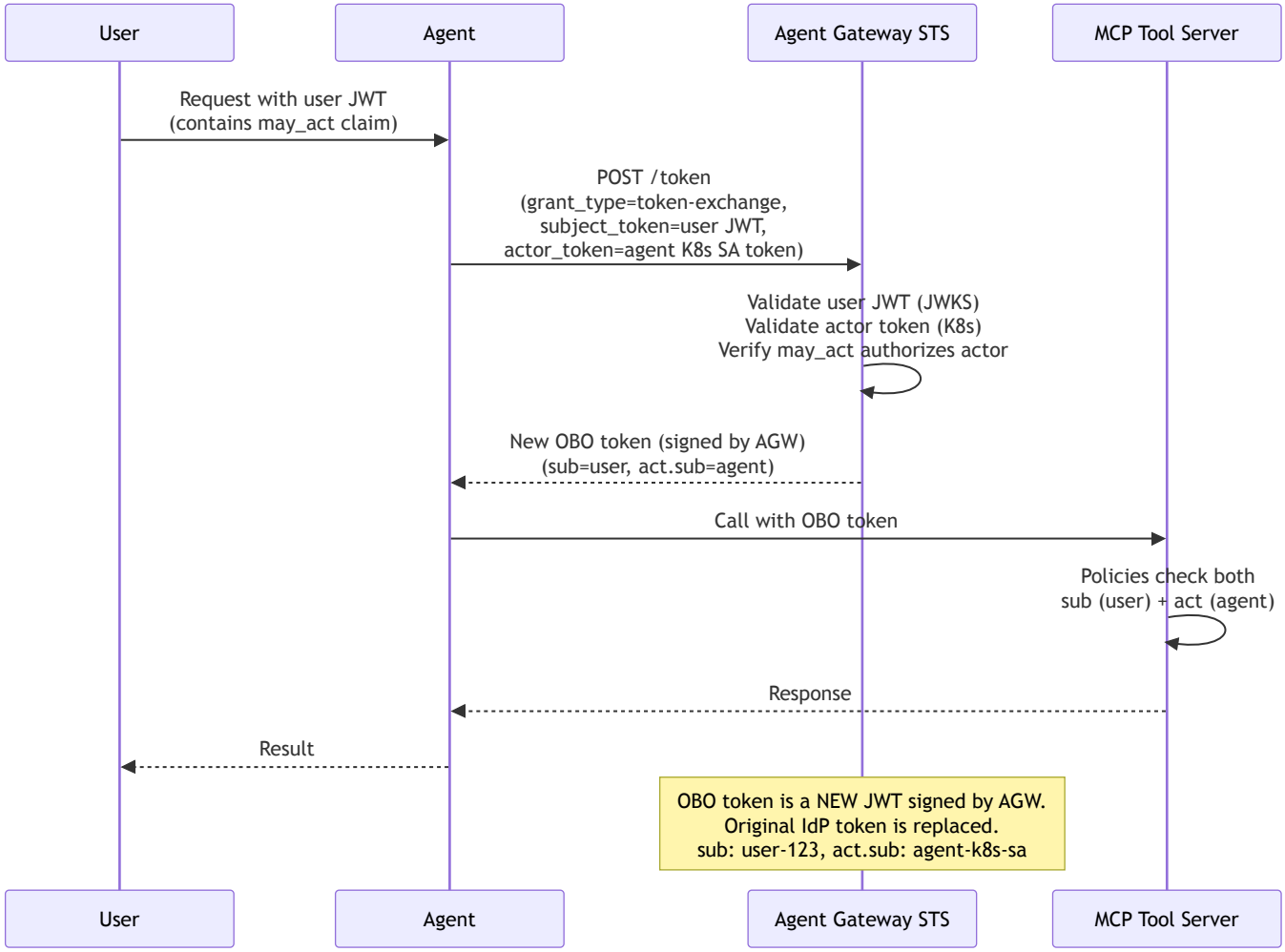


< Table of Contents

Flow 2a: OBO Delegation (Dual Identity)

Agent exchanges the user's JWT for a delegated OBO token via RFC 8693 Token Exchange. The user's JWT must include a `may_act` claim authorizing the agent. The STS validates both the user JWT and the agent's K8s service account token, then issues a new JWT (signed by Agent Gateway) containing both `sub` (user) and `act` (agent). Downstream services trust the Agent Gateway issuer and can enforce policies on both identities.

> OBO Token Exchange | About OBO & Elicitations
> Helm tokenExchange values

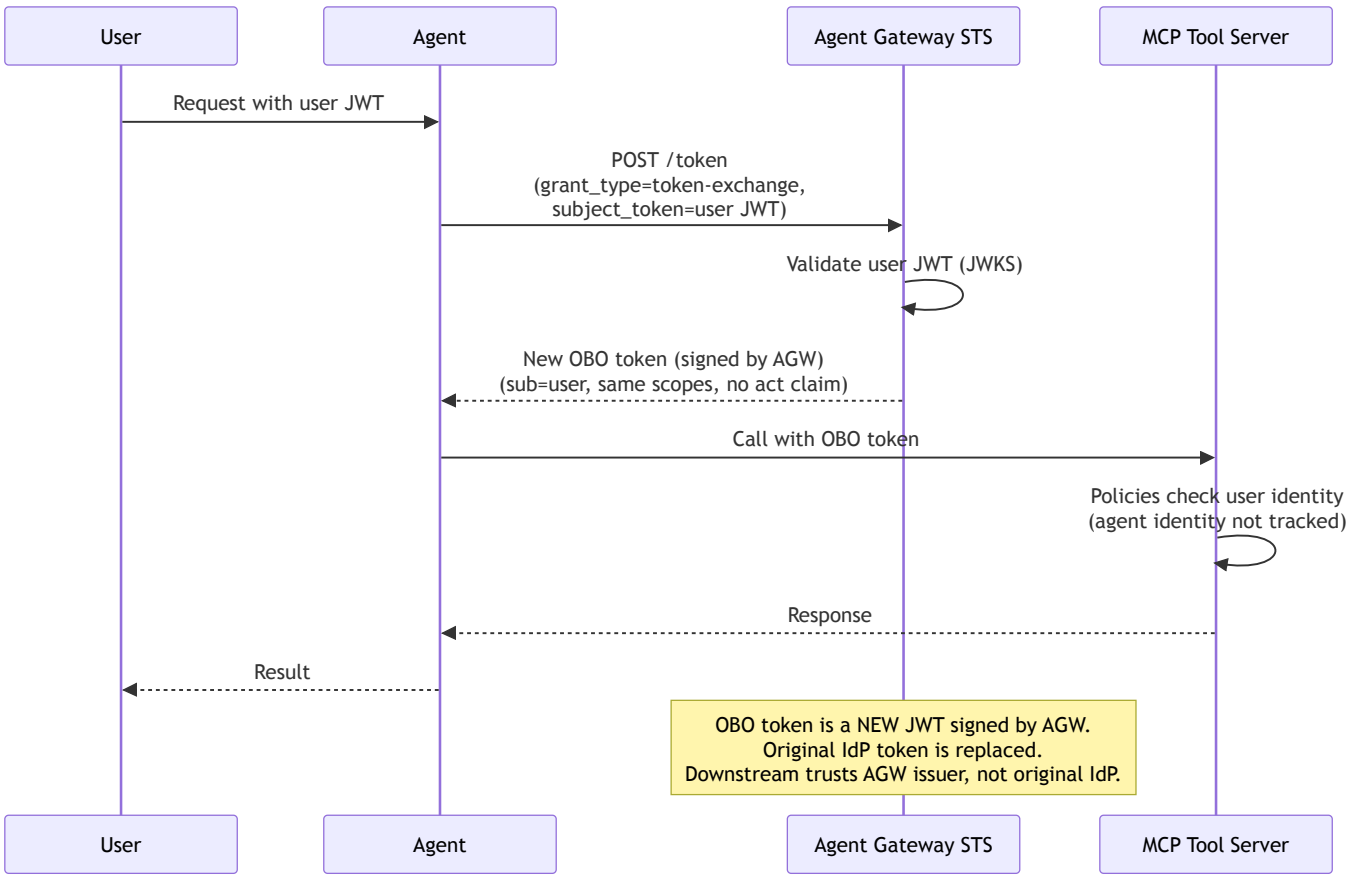


< Table of Contents

Flow 2b: OBO Impersonation (Token Swap)

Agent exchanges the user's JWT for a new OBO token via RFC 8693, but without an actor token. The STS validates the user JWT, then issues a new JWT (signed by Agent Gateway) with the same `sub` and scopes — no `act` claim. Downstream services trust the Agent Gateway issuer and see only the user's identity. The original IdP token is replaced, keeping user identity consistent without passing IdP tokens through the stack.

> OBO Token Exchange | About OBO & Elicitations
> Helm tokenExchange values

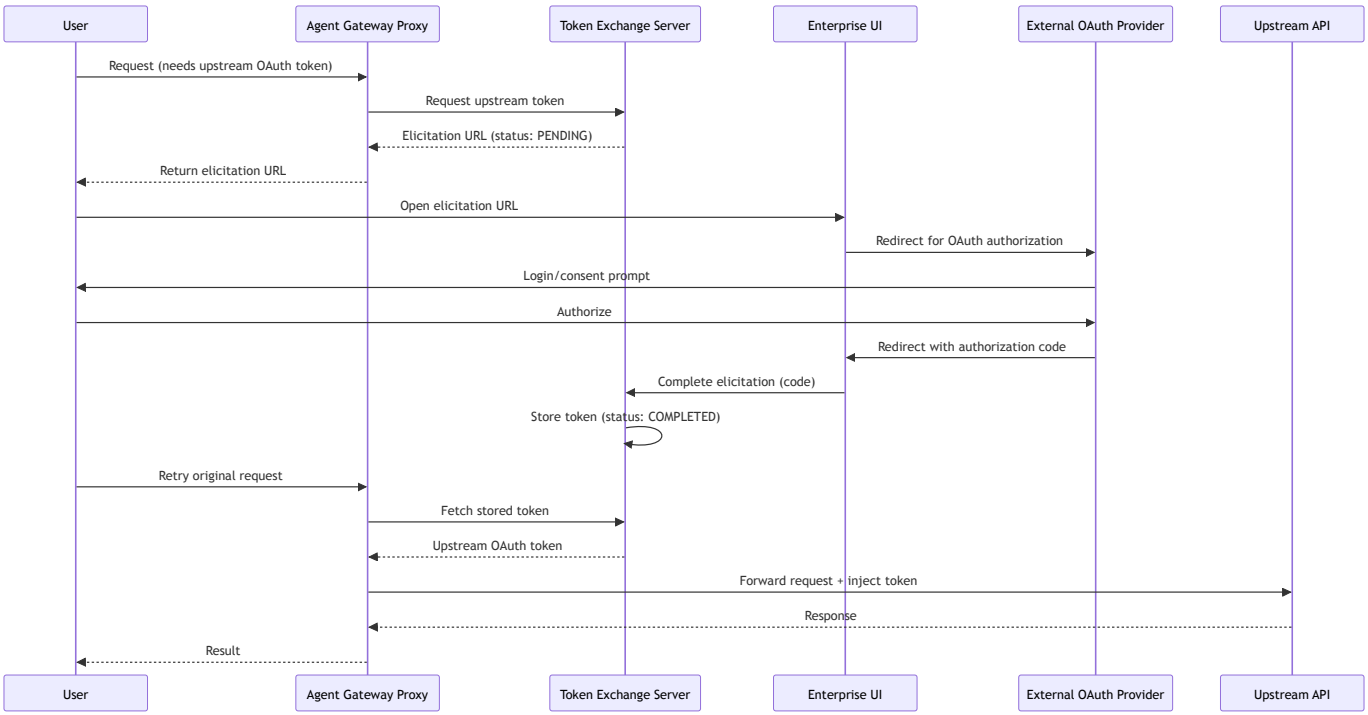


< Table of Contents

Flow 3: Elicitation (Credential Gathering for Upstream APIs)

When the agent needs to call an upstream API requiring OAuth credentials that don't exist yet. The gateway returns an elicitation URL; the user completes an out-of-band OAuth flow to provide the credentials.

> Elicitations | About OBO & Elicitations
> TokenExchangeMode

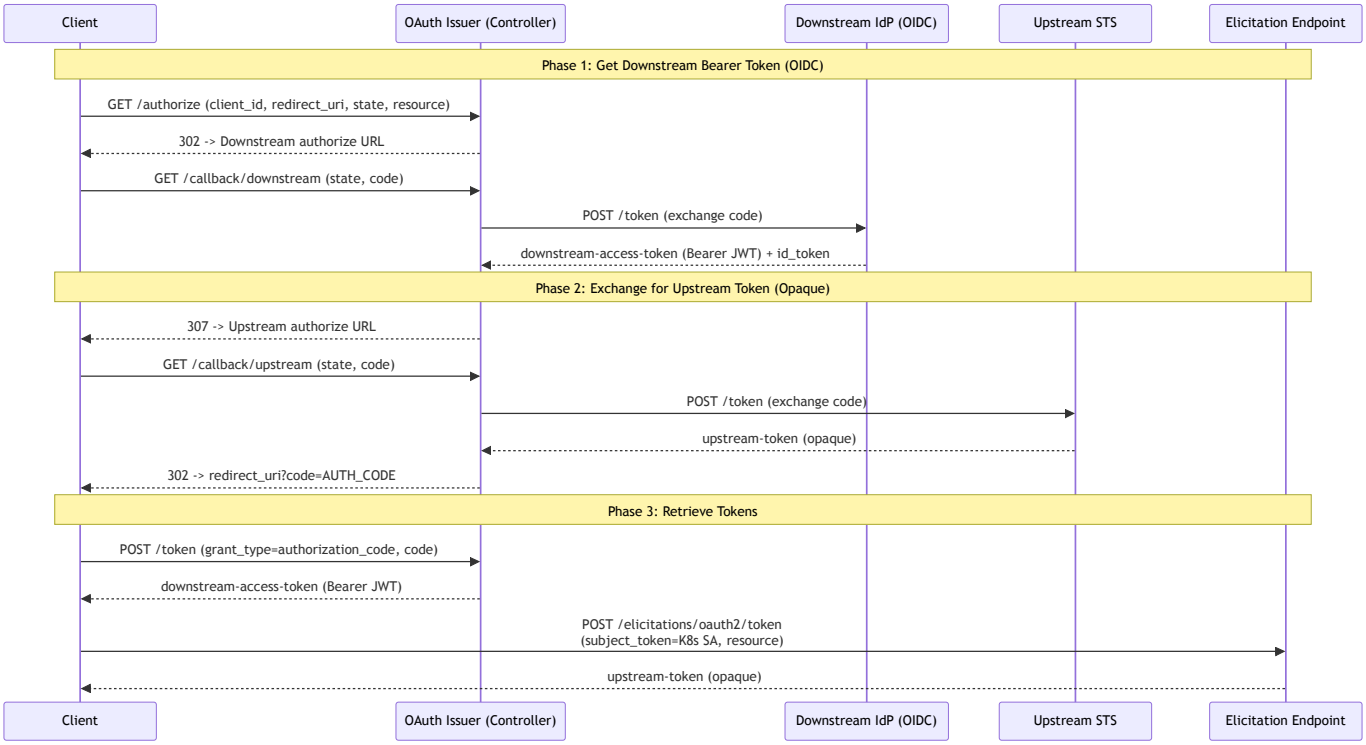


< Table of Contents

Flow 4: Double OAuth Flow (OIDC Bearer → Upstream Token Exchange)

User authenticates via OIDC (gets bearer JWT), then that token is exchanged for a different upstream token (could be opaque). Combines downstream and upstream OAuth in a single automated flow.

> OBO Token Exchange | Elicitations

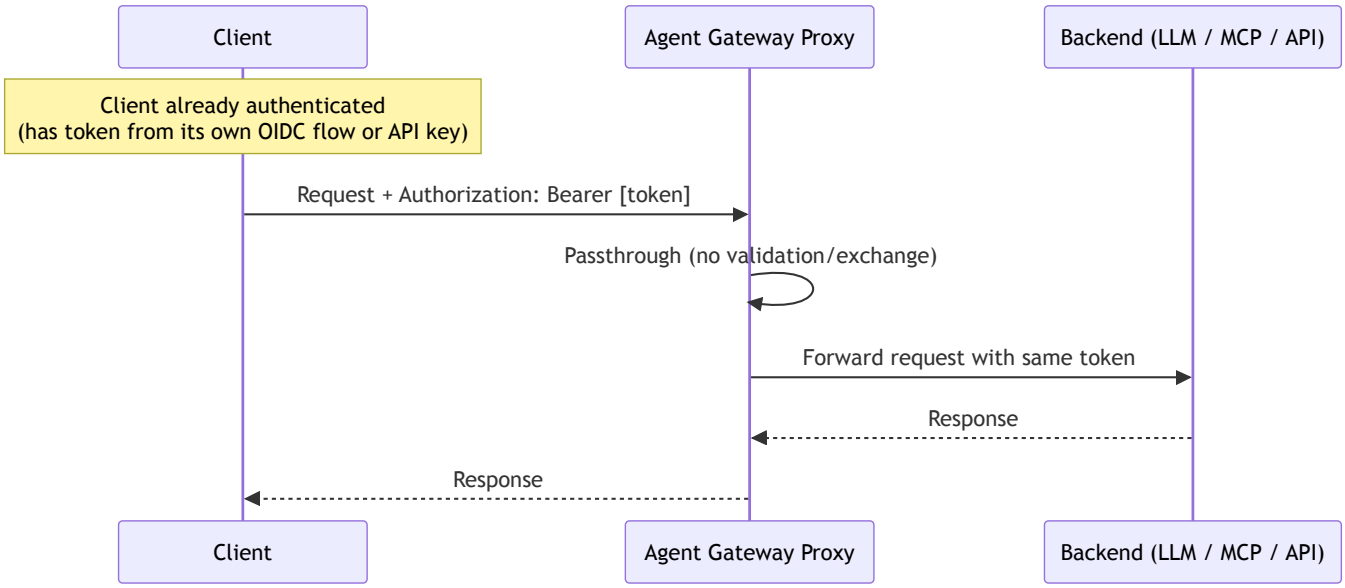


< Table of Contents

Flow 5: Passthrough Token

Client already has the correct token (from its own OIDC flow or API key). Gateway forwards it directly to the backend — no validation or exchange performed.

> API Keys — Passthrough Token
> AIBackend

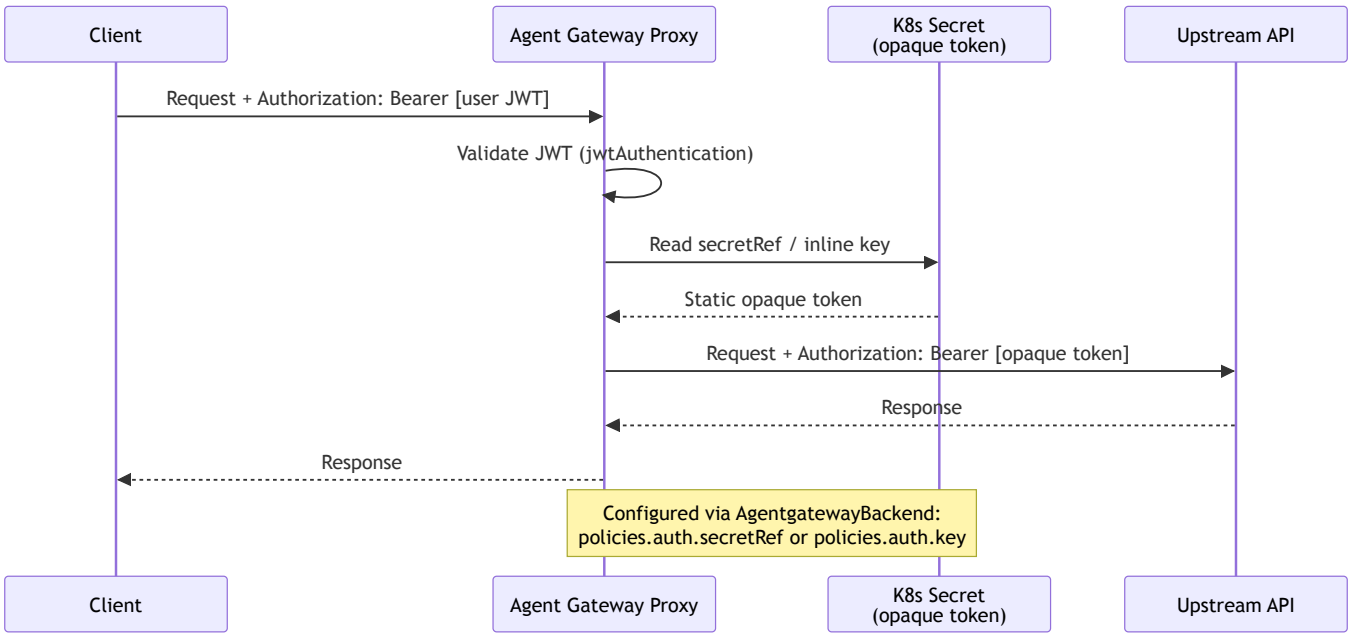


< Table of Contents

Flow 6: Static Secret Injection (Shared Credential)

Gateway validates inbound auth (JWT or API key), then replaces it with a static backend credential from a Kubernetes secret. All users share the same upstream token.

> API Keys — Manage API Keys
> AIBackend

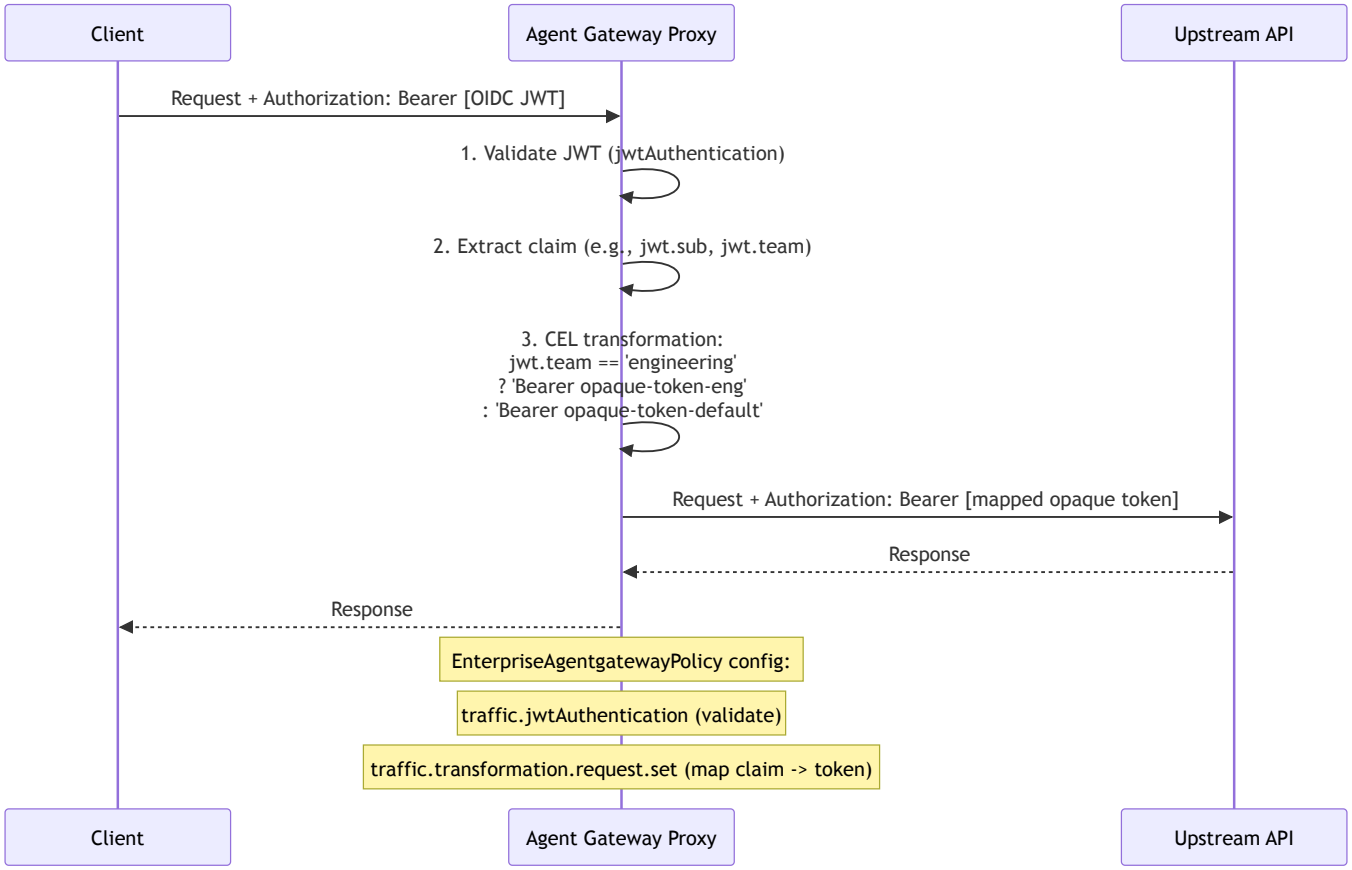


< Table of Contents

Flow 7: Claim-Based Token Mapping (JWT Claim → Static Opaque Token)

Validate the inbound OIDC JWT, inspect a claim (sub, team, tier), then use a CEL transformation to inject a per-user or per-group static opaque token. Enables differentiated backend access based on identity attributes.

> CEL Transformations | JWT Auth for MCP Services
> EnterpriseAgentgatewayPolicyTraffic

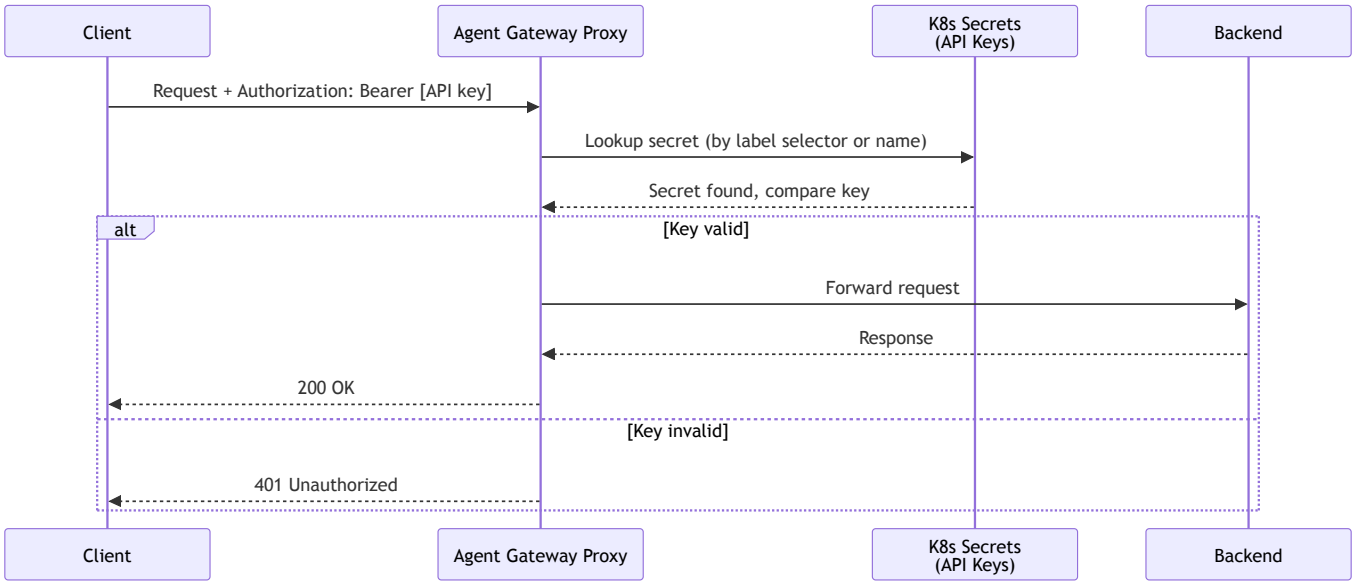


< Table of Contents

Flow 8: API Key Auth (Inbound)

Clients authenticate with a static API key instead of OIDC. Gateway validates the key against Kubernetes secrets (by label selector or name).

> API Key Auth
> APIKeyAuthentication

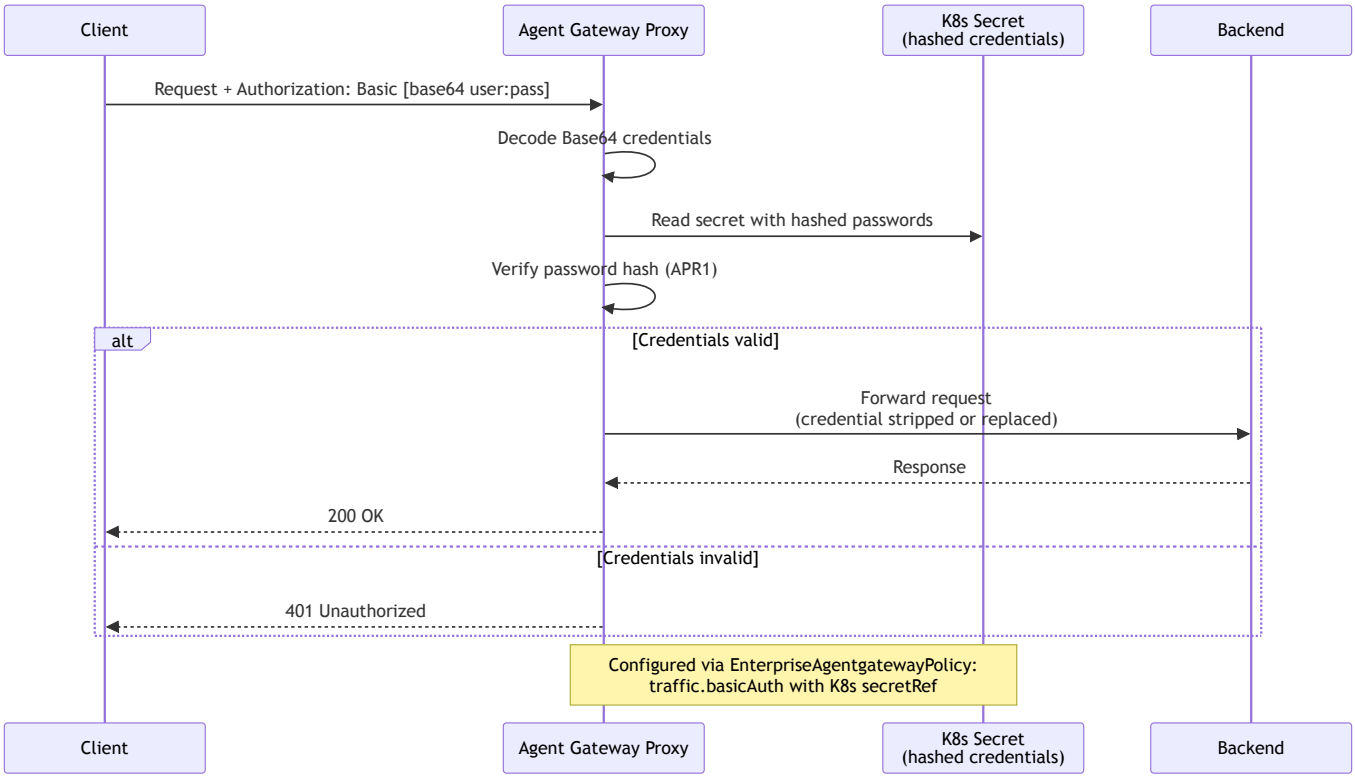


< Table of Contents

Flow 9: Basic Auth (RFC 7617)

Clients authenticate with username and password (Base64-encoded in the Authorization header). Gateway validates credentials against hashed values stored in Kubernetes secrets. Useful for legacy integrations or simple service-to-service auth.

> Basic Auth
> BasicAuthentication

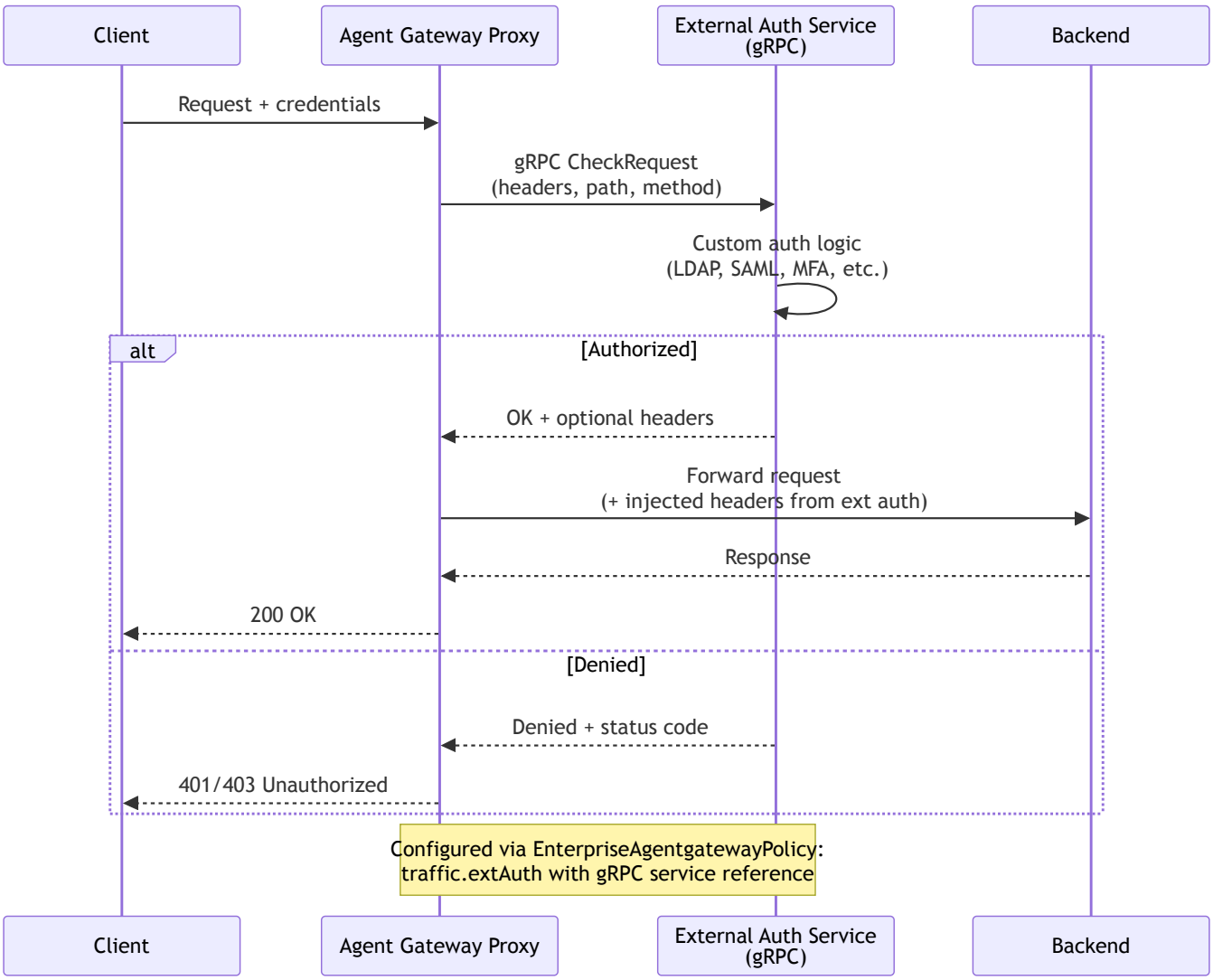


< Table of Contents

Flow 10: BYO External Auth (gRPC Ext Auth Service)

Delegate authentication to your own external authorization service via gRPC. The gateway sends auth check requests to your service, which returns allow/deny decisions. Supports custom logic, enterprise IdPs, or multi-factor checks.

> External Auth
> EnterpriseAgentgatewayExtAuth

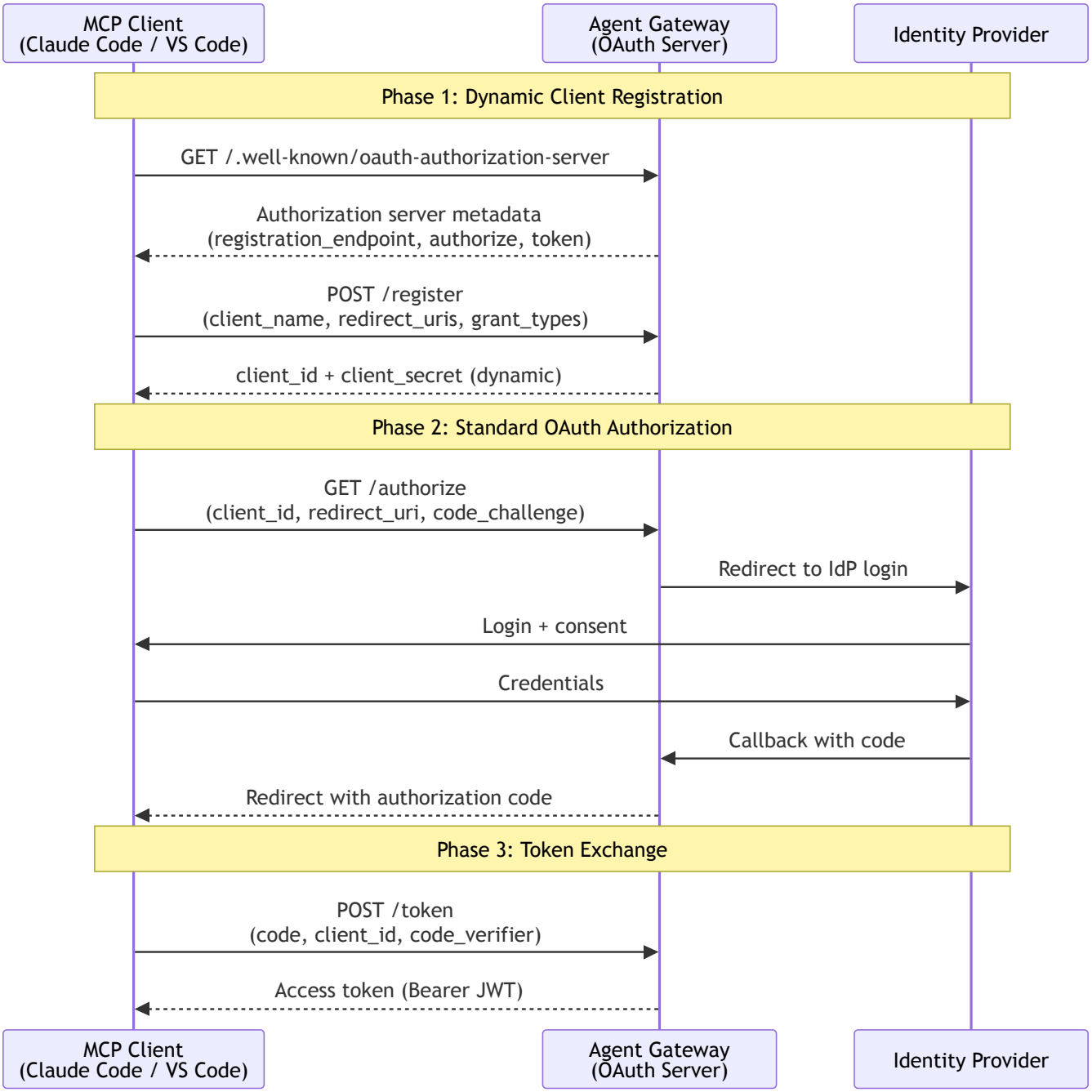


< Table of Contents

Flow 11: MCP OAuth with Dynamic Client Registration

MCP clients (like Claude Code, VS Code extensions) that don't have pre-registered OAuth credentials use Dynamic Client Registration (DCR) to register themselves, then complete a standard OAuth flow. Enables zero-configuration MCP client onboarding.

> About MCP Auth | Set up Keycloak for MCP Auth

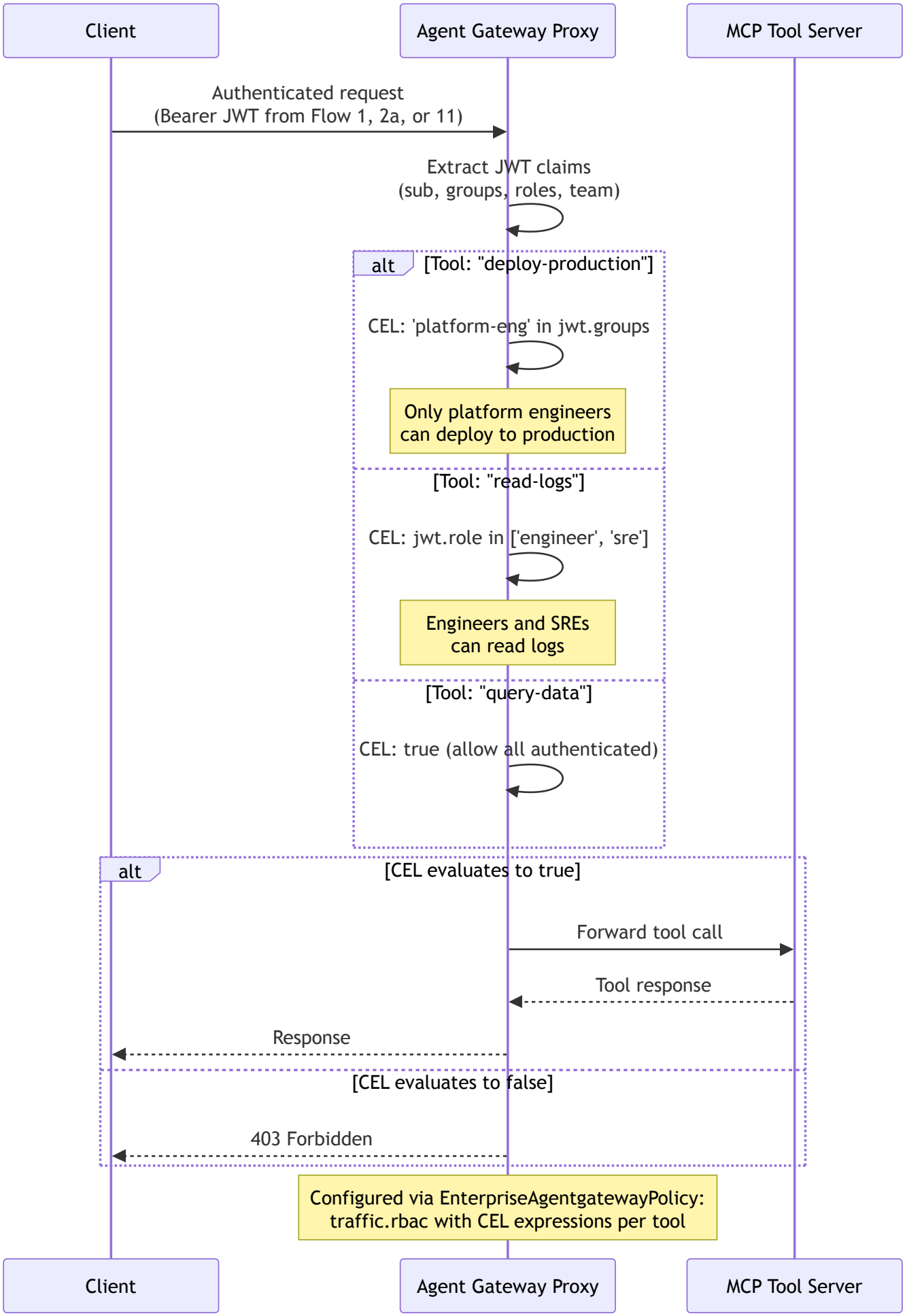


< Table of Contents

Flow 12: RBAC Tool-Level Access Control

After authentication (via any flow), apply per-tool authorization using CEL expressions evaluated against JWT claims. Controls which users or groups can invoke specific MCP tools.

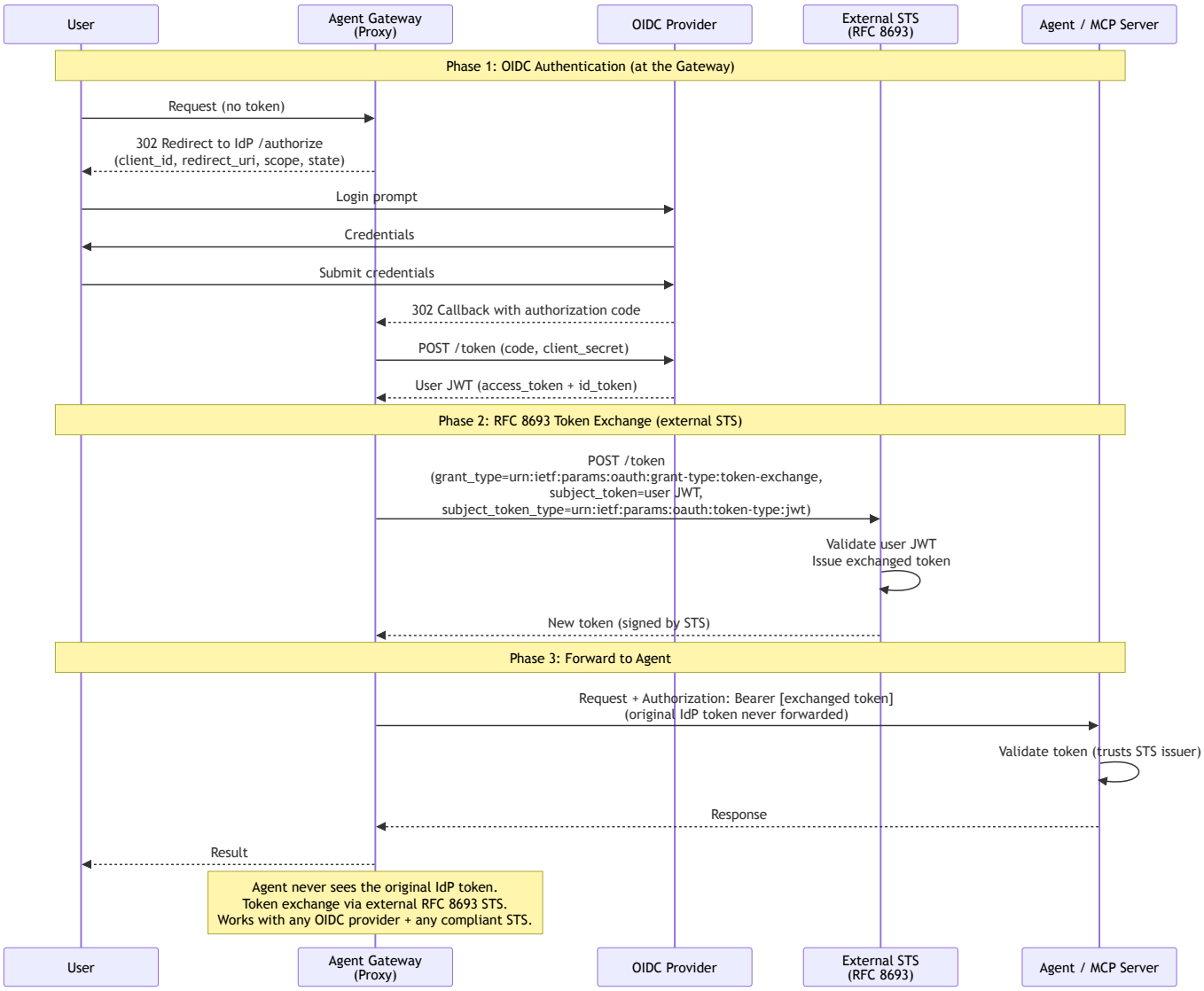
> Control Access to Tools
> EnterpriseAgentgatewayPolicyTraffic (rbac)


Client
Agent Gateway Proxy
MCP Tool Server
Authenticated request
(Bearer JWT from Flow 1, 2a, or 11)
Extract JWT claims
(sub, groups, roles, team)
alt
[Tool: "deploy-production"]
CEL: 'platform-eng' in jwt.groups
Only platform engineers
can deploy to production
[Tool: "read-logs"]
CEL: jwt.role in ['engineer', 'sre']
Engineers and SREs
can read logs
[Tool: "query-data"]
CEL: true (allow all authenticated)
alt
[CEL evaluates to true]
Forward tool call
Tool response
Response
[CEL evaluates to false]
403 Forbidden
Configured via EnterpriseAgentgatewayPolicy:
traffic.rbac with CEL expressions per tool
Client
Agent Gateway Proxy
MCP Tool Server

Flow 13: Gateway-Mediated OIDC + Token Exchange

Agent Gateway handles OIDC authentication, then exchanges the IdP token with an external RFC 8693 Security Token Service (STS) before forwarding to the agent. The agent never sees the original IdP token — it trusts only the STS issuer. Decouples the IdP from downstream services and works with any compliant STS.

> OBO Token Exchange | Set up JWT Auth
> Helm tokenExchange values

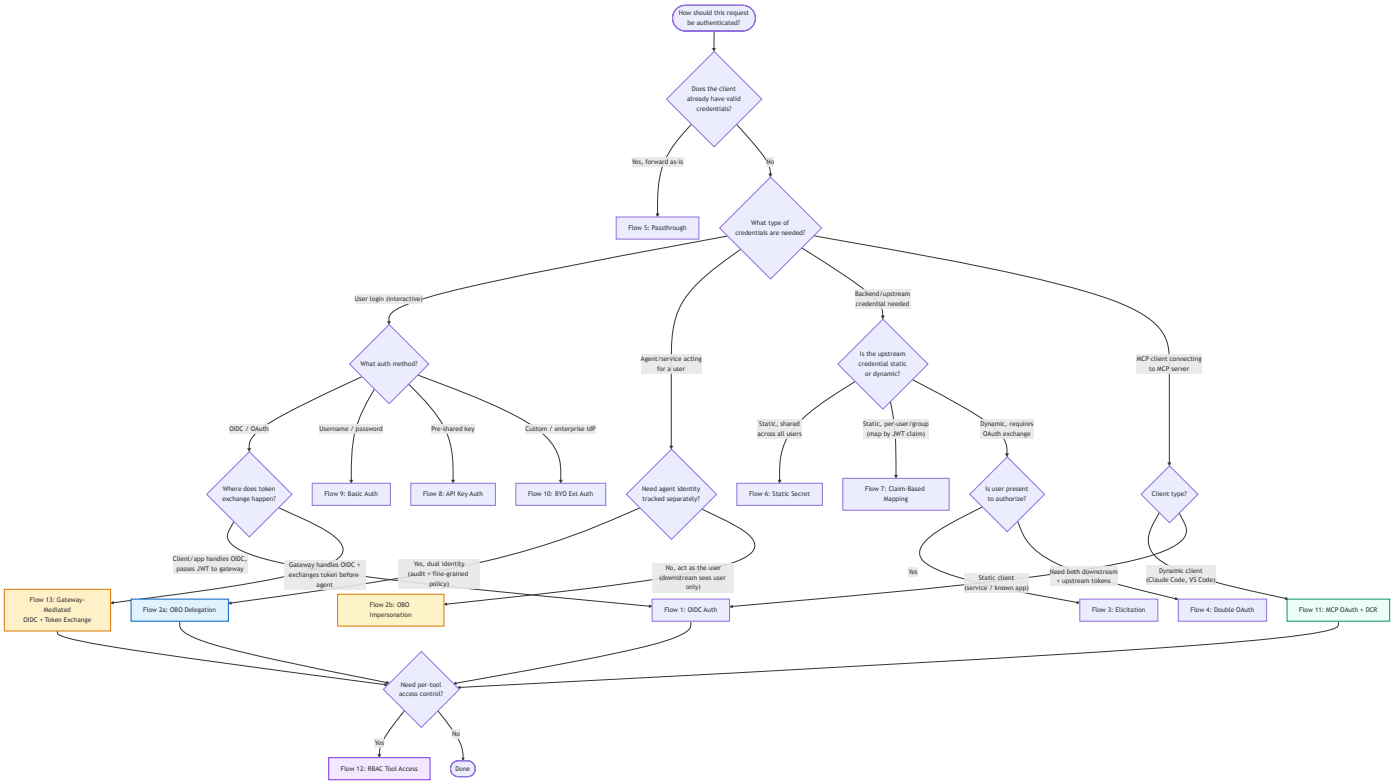


< Table of Contents

Decision Flowchart: How Should This Request Be Authenticated?

flowchart TD

> Security Overview | OBO & Elicitations | External Auth | MCP Auth
> Enterprise API Reference



< Table of Contents